

CS2111
Web Application Project

FlowRose

By: Kamar Bdair
Layan Ziyadah
Faisal Shamsi

Introduction

FlowRose is an e-commerce website designed to simplify the process of purchasing and delivering flower bouquets online. The platform addresses the common challenges associated with buying flowers from physical stores, such as limited selection, inconvenient store hours, and time-consuming shopping experiences. By offering an intuitive online interface, FlowRose enables customers to browse, personalize, and order flower arrangements for various occasions—all from the comfort of their homes. The project was developed as a collaborative effort, integrating front-end design, back-end functionality, and user experience optimization to create a seamless and visually appealing platform for flower gifting in Saudi Arabia.

Project Objectives

The primary objectives of the FlowRose project were:

- To create a user-friendly online platform for flower bouquet selection and delivery.
- To provide a wide range of bouquet categories, including roses, mixed flowers, and seasonal collections.
- To integrate luxury add-ons such as perfumes, skincare, and candles to enhance gifting options.
- To ensure a responsive and visually consistent design across all devices. To implement a functional shopping cart and secure checkout process.

Methodology

The project followed a structured development approach:

- Technology Stack: Front-end: HTML, CSS, JavaScript
- Back-end: Server-side scripting and database integration
- Responsive design frameworks for mobile and desktop compatibility

Team Roles and Contributions:

- Faisal**: Managed checkout processes and overall styling/responsive layout.
- Layan**: Focused on page layouts, bouquet detail pages, and shopping cart features.
- Kamar**: Worked on back-end/database connections and designed the home and bouquet listing pages.

Design Philosophy: Emphasis was placed on aesthetic appeal, ease of navigation, and visual consistency to reflect the elegance of flower gifting.

Key Features

FlowRose includes several core features to enhance the user experience:

- Home Page**: Displays featured bouquets and seasonal collections.
- Category Browsing**: Users can filter bouquets by type (e.g., Roses, Mixed, Seasonal).
- Shopping Cart**: Allows users to add items, review selections, and proceed to checkout.
- Luxury Add-ons**: Optional premium gifts such as perfumes, makeup, and candles from top brands.
- Personalization**: Options to include gift messages and special delivery instructions.
- Responsive Design**: Fully functional on both desktop and mobile devices.

1. Backend Architecture Overview

FlowRose's backend appears to be built around a **type-safe API layer** using **tRPC** and a **schema-validated input system** via **Zod**. This is a modern approach that reduces runtime errors, enforces clean boundaries between frontend and backend, and makes the codebase scalable as the business grows.

At the center of the backend is a single exported router:

```
export const appRouter = router({
  system: systemRouter,
  auth: router({...}),
  products: router({...}),
  categories: router({...}),
});
export type AppRouter = typeof appRouter;
```

This confirms that the backend follows a **domain-based modular router design**, where each business domain (authentication, products, categories) is isolated into its own sub-router. Everything is then composed into the appRouter, which becomes the **source of truth API contract** for the entire application. Why this matters for FlowRose

FlowRose is not just a static catalog — it includes bouquets, add-ons, collections, and likely order/delivery workflows. A modular router design ensures that:

- Product browsing features do not get tangled with auth logic.

- Seasonal/event collections can expand without breaking other endpoints.
- Future additions (orders, payments, delivery scheduling) can be introduced cleanly.

2. Core Imports and Their Roles

Your snippet imports multiple internal utilities and 3rd-party libraries:

```
import { COOKIE_NAME } from "@shared/const";
import { getSessionCookieOptions } from "../server/_core/cookies";
import { systemRouter } from "../server/_core/systemRouter";
import { publicProcedure, protectedProcedure, router } from "../server/_core/trpc";
import { z } from "zod";
import * as db from "../server/db";
```

Each import signals a specific architectural layer:

2.1 Shared constants

`COOKIE_NAME` is imported from a shared constants module. The `@shared` alias implies a monorepo or alias config where constants are shared across server and client. This avoids cookie name mismatches between frontend and backend.

Benefit:

If the cookie name changes, it changes everywhere at once, preventing auth bugs in production.

2.2 Cookie core utilities

`getSessionCookieOptions(ctx.req)` centralizes cookie configuration. Instead of duplicating cookie flags (secure, sameSite, domain, path), the backend uses a single helper.

Expected behavior:

- detect environment (dev vs prod)
- set secure: true for HTTPS only
- enforce sameSite rules for CSRF resistance
- set correct domain/path for subdomains (if used)

Benefit:

Consistent cookie behavior ensures customers don't randomly "lose login" depending on browser or environment.

2.3 systemRouter

systemRouter is mounted at:
system: systemRouter

Although not shown, most production tRPC apps include system-level procedures such as:

- healthcheck
- version
- status
- possibly ping

Benefit for FlowRose:

Order/delivery apps need reliability. A system router makes it easy to monitor uptime and performance.

2.4 tRPC core

```
import { publicProcedure, protectedProcedure, router } from "../server/_core/trpc";
```

This indicates a custom tRPC initialization file that likely:

- creates context (ctx) that includes req, res, user, etc.
- defines publicProcedure (no auth required)
- defines protectedProcedure (auth required)

Even though your snippet uses only publicProcedure, the presence of protectedProcedure strongly suggests other parts of the codebase enforce authentication, likely for:

- order creation
- saved addresses
- profile data
- admin operations

2.5 Zod validation

Zod is used to validate user inputs:

```
.input(z.object({ type: z.enum(["bouquet", "addon"]) })))
```

In tRPC, Zod schemas serve two roles:

1. runtime validation (reject invalid inputs)
2. compile-time typing (frontend gets exact types)

Why FlowRose benefits:

For e-commerce flows, bad inputs can cause:

- product ID mismatches
- pricing errors
- security risks (SQL injection attempts)
Zod hard-blocks these at API boundaries.

2.6 Database layer (db)

Your router procedures never query the database directly — they delegate to:
`await db.getAllProducts()`

This implies a **data-access abstraction**, probably with:

- raw SQL
- Prisma
- Drizzle ORM
- or another query builder

Advantages:

- Routers remain thin and readable.
- DB logic is reusable between procedures.
- Schema/table changes are isolated to db functions.

3. appRouter Design and Domain Grouping

The appRouter is organized by business functions:

1. System
2. Auth
3. Products
4. Categories

3.1 Composition pattern

FlowRose uses router nesting:

```
auth: router({...})
products: router({...})
categories: router({...})
```

This creates API namespaces such as:

- auth.me
- auth.logout
- products.list
- products.byType
- categories.list

4. Products Router (Deep Dive)

Products are central for FlowRose because the platform sells bouquets + gifts.

```
products: router({
  list: publicProcedure.query(async () => { ... }),
  byType: publicProcedure.input(...).query(async ({ input }) => { ... }),
  bestsellers: publicProcedure.query(async () => { ... }),
  byId: publicProcedure.input(...).query(async ({ input }) => { ... }),
}),
```

4.1 products.list

Purpose: Retrieves all products.

Possible internal behavior in db layer:

- join product table with categories
- include pricing, images, stock status

Use cases:

- admin dashboards
- search index generation
- “browse everything” pages

4.2 products.byType

```
.input(z.object({ type: z.enum(["bouquet", "addon"]) })))  
.query(async ({ input }) => db.getProductsByType(input.type));
```

This is a key endpoint for FlowRose’s business model.

It forces the domain logic that FlowRose has two product types:

1. **bouquet** (main purchasable)
2. **addon** (gift extras)

Why strict enum is valuable:

- prevents typos like "bouquets" causing empty results
- ensures frontend never requests invalid product types
- makes the API self-documenting

Frontend flow this supports:

1. Customer browses bouquets.
2. Chooses one bouquet.
3. Frontend loads add-ons via byType({ type: "addon" }).

4. User bundles gifts before checkout.

4.3 products.bestsellers

Purpose: Retrieves popular items.

Most likely db logic:

- sort by sales volume table
- filter by availability

Challenges and Learning Outcomes

During development, the team encountered several challenges:

-Design Consistency: Maintaining a uniform visual theme across all pages required careful attention to detail and iterative testing.

-Back-End Integration: Connecting the front-end interface with the database and ensuring secure transaction processing posed technical hurdles.

-User Experience Optimization: Balancing aesthetic design with functional efficiency was an ongoing focus.

-Learning Outcomes: The project provided hands-on experience in full-stack web development, collaborative teamwork, and problem-solving in a real-world context. It also highlighted the importance of user-centered design in e-commerce platforms.

Results

The final product is a fully functional e-commerce website that includes:

- A visually engaging home with bouquet displays.
- A categorized bouquet listing page.
- A working shopping cart and checkout system.
- Mobile-responsive layouts.
- Integration of luxury gift add-ons.

Limitations

Despite its successes, the project has certain limitations:

- Payment Gateway: Currently, the payment system is simulated and not integrated with real financial gateways.
- Inventory Management: The back-end does not yet include real-time inventory tracking.
- Localization: The platform is currently optimized for Saudi Arabia, with limited multi-language support.
- Scalability: The database structure may require optimization for large-scale user traffic.

Conclusion and Future Work

FlowRose successfully delivers an online flower gifting solution that combines elegance, convenience, and functionality. The platform meets its core objectives and provides a solid foundation for future enhancements.

Future Recommendations:

- Integrate a secure payment gateway (e.g., PayPal, local bank options).
- Implement an admin panel for inventory and order management.
- Expand delivery areas beyond Saudi Arabia.
- Introduce AI-based bouquet recommendations based on occasion and user preferences.
- Develop a mobile app version for increased accessibility.